TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Colivi: Una app de alojamientos para estudiantes.



Curso: 2026/2027

Alumno/a:
Víctor Vallejo Uroz

Director/es:
Luis Iribarne Martínez

Texto de la dedicatoria

ÍNDICE GENERAL

Página

Resumen y Abstract XII

1. Introducción 1

1.1. Justificación . . . 1
1.2. Objetivos Y Estructura . . . 2
1.3. Planificacion . . . 4
1.4. Apps Similares . . . 6
1.5. Contenido de la memoria . . . 10

2. Herramientas y metodologías 13

2.1. Metodologías usadas . . . 14
2.1.1. Revisión de enfoques metodológicos . . . 14
2.1.2. Análisis comparativo y selección para el proyecto . . . 16
2.2. Herramientas usadas . . . 18
2.2.1. Entornos de desarrollo y asistencia técnica . . . 18
2.2.2. Lenguajes, marcos de trabajo y servicios . . . 19
2.2.3. Almacenamiento y persistencia de datos . . . 20
2.2.4. Control de versiones, infraestructura y utilidades auxiliares . . . 20

3. Apartados del CUERPO DEL DOCUMENTO 23

4. Apartado BIBLIOGRAFIA 27

4.1. Como citar . . . 27
4.2. Apartado referencias . . . 27
4.3. Archivos .bib . . . 28
BIBLIOGRAFÍA . . . 28

A. Contenido de una entrada en .bib 31

B. Tipos de referencias 33

ÍNDICE DE FIGURAS

1.1. Estructura técnica global y arquitectura de la plataforma Colivi 4
1.2. Planificación Temporal General 5
1.3. Planificación Temporal de la implementación 6
1.4. Interfaces de las principales soluciones analizadas en el mercado 8

2.1. Diferencia de éxito entre metodologías ágiles y tradicionales en cascada 15
2.2. Tablero Kanban representativo del flujo de desarrollo ágil 16
2.3. Ciclo iterativo de la metodología TDD (*Test-Driven Development*) 17
2.4. Entornos de desarrollo utilizados en el proyecto. Elaboración propia 19
2.5. Sistema gestor de bases de datos PostgreSQL 20
2.6. Herramientas auxiliares empleadas para diseño de interfaz, modelado y verificación de API 21

3.1. Estructura clásica de un articulo de investigación 24
3.2. Estructura de documentos de un proyecto técnico 24
3.3. Planificación del proyecto 25

ÍNDICE DE TABLAS

1.1. Comparativa de características entre plataformas analizadas y Colivi 7

A.1. Campos de una entrada en archivo de bibliografía .bib 32

B.1. Tipos posibles de elementos en bibligrafía con natbib 34

LISTADOS

ABREVIATURAS

ESI	Escuela Superior de ingeniería
InSo	Ingeniería del Software
SBSE	Search based software engineering
TFG	Trabajo fin de grado
UAL	Universidad de Almería

RESUMEN Y ABSTRACT

En este proyecto se ha llevado a cabo el desarrollo de Colivi, una plataforma web orientada a resolver los problemas comunes en la busqeuda de alojamiento compartido y la gestión de la convivencia en pisos de estudiantes.

Para solucionar esto, se ha implementado una arquitectura basada en un backend con Spring boot (java 21) y PostgreSQL, junto con un frontend interactivo desarrollado en React. Además, se ha integrado un microservicio en Node.js que implementa el protocolo MCP (Model Context Protocol) para incorporar un asistente conversacional capaz de consultar el estado de la vivienda, recomendar anuncios o resumir la actividad mediante modelos de lenguaje.

El objetivo era consolidar buenas prácticas de ingeniería de software mediante principios SOLID y diseño guiado por el dominio (DDD), aprendiendo a resolver problemas reales de concurrencia, sincronización de datos y gamificación de tareas dando como resultado un sistema integral que cubre desde el primer contacto entre inquilino y propietario hasta el día a día dentro del hogar.

This project presents the design and implementation of Colivi, a web platform created to address common challenges in finding shared accommodations and managing household life in student flats. The main motivation behind the project stems from my personal experience during an Erasmus mobility program, where I faced significant hurdles searching for rooms from abroad, dealing with uncoordinated landlords, and managing daily domestic chores and living friction with flatmates.

To solve these issues, the system relies on a backend built with Spring Boot (Java 21) and PostgreSQL, alongside a responsive frontend developed with React. Additionally, an external Node.js microservice implementing the Model Context Protocol (MCP) was integrated to power an intelligent assistant capable of querying household chores, recommending accommodation options, and summarizing inbox activity using language models.

The primary goal was to strengthen software engineering best practices through SOLID principles and domain-driven design, learning to tackle real-world concurrency challenges, data synchronization, and gamification mechanisms. The final outcome is a comprehensive platform that supports users throughout the entire flat-sharing lifecycle, from initial room discovery to day-to-day household organization.

1 Introducción

Compartir piso ha dejado de ser algo exclusivo de estudiantes que se van de casa por primera vez para convertirse en la norma entre jóvenes trabajadores y universitarios de toda Europa. Aunque antes se veía como una etapa corta de un par de cursos, la realidad del mercado inmobiliario actual obliga a alargar esta forma de vivir durante años. El aumento de los precios del alquiler y la falta de oferta en las ciudades medianas y grandes han hecho que alquilar una habitación sea prácticamente la única salida viable frente a pagar un piso entero.Servirá de ayuda para la realización de los trabajos académicos que, durante la formación universitaria, constituyen una actividad permanente.

A este escenario se suma la movilidad internacional. Cada año miles de estudiantes se desplazan con becas como la Erasmus, encontrándose con la urgencia de buscar techo en ciudades que no conocen y en idiomas que muchas veces no dominan. Gestionar un alquiler a ciegas, sin poder ver la habitación en persona y teniendo que fiarse de fotos sueltas en internet, añade una capa extra de estrés e incertidumbre que complica el proceso desde el primer día.

Sin embargo, a pesar de este crecimiento en la demanda, el sector sigue arrastrando fricciones estructurales. Los canales tradicionales de búsqueda continúan dependiendo en gran medida de portales genéricos de compraventa, grupos informales en redes sociales o intermediarios sin garantías formales. Este ecosistema disperso genera problemas recurrentes: falta de transparencia en las condiciones contractuales, riesgos de fraude en la reserva a distancia, duplicidad de comunicaciones y una notable asimetría informativa entre propietarios e inquilinos antes de formalizar cualquier acuerdo.

A estas complicaciones de acceso se suma una segunda fase no menos crítica: la gestión de la convivencia diaria. Una vez formalizada la estancia, surgen los roces habituales de la vida comunitaria, tales como el reparto desequilibrado de las tareas del hogar, el seguimiento de las responsabilidades domésticas o la falta de claridad en las normas internas del piso. La mayoría de estas dinámicas se siguen gestionando con métodos rudimentarios —notas adhesivas en zonas comunes o cadenas de mensajes caóticas en aplicaciones de mensajería instantánea—, herramientas que carecen de seguimiento objetivo y que con frecuencia derivan en tensiones evitables entre compañeros.

1.1 Justificación

La concepción de este Trabajo de Fin de Grado responde a la necesidad de abordar una problemática recurrente en el ámbito del alojamiento compartido para estudiantes universi-

tarios, superando el alcance de un ejercicio meramente teórico. La identificación de estas fricciones parte del análisis del contexto de los programas de movilidad internacional (como las estancias Erasmus), donde la búsqueda de habitación a distancia pone de manifiesto una acusada asimetría de información: anuncios con especificaciones incompletas, tiempos de respuesta prolongados por parte de los arrendadores, falta de transparencia en las condiciones pactadas y un clima general de incertidumbre en la formalización del alquiler.

A esta complejidad logística inicial se superpone la fase operativa de la convivencia dentro del inmueble. El régimen de uso compartido de espacios comunitarios genera tensiones cotidianas derivadas de la descoordinación en la distribución de tareas y la ausencia de mecanismos formales para el seguimiento de las responsabilidades domésticas. Los cauces informales habituales —tales como aplicaciones de mensajería instantánea o notas físicas— evidencian una rápida pérdida de eficacia organizativa, al carecer de trazabilidad objetiva y propiciar un reparto desequilibrado de las obligaciones comunes.

Al contrastar este escenario con el software existente en el mercado, se constata una fragmentación funcional significativa: las soluciones actuales se concentran de forma aislada en la intermediación publicitaria o en la gestión puntual de tareas del hogar, sin existir una integración continua entre la reserva y la vida real dentro del alojamiento. Con el propósito de mitigar esta desconexión, el presente proyecto plantea una plataforma integral que unifica ambos ciclos. Dicha solución articula una arquitectura backend robusta basada en Spring Boot, una interfaz reactiva orientada a la experiencia de usuario y un módulo de asistencia contextual sustentado en el protocolo MCP (Model Context Protocol), cubriendo de manera homogénea desde el proceso inicial de arrendamiento hasta la gestión activa de la convivencia comunitaria.

1.2 Objetivos Y Estructura

El objetivo principal de este trabajo es diseñar y desarrollar Colivi, una plataforma web completa pensada para resolver los problemas habituales en la búsqueda de alojamiento compartido y simplificar la gestión cotidiana dentro de una vivienda de estudiantes o jóvenes profesionales. Se busca ofrecer una solución práctica e intuitiva que acompañe al usuario en todo el proceso: desde la búsqueda inicial de habitación y el contacto directo con el casero, hasta el reparto organizado de las tareas del hogar mediante dinámicas participativas una vez iniciada la convivencia. Para alcanzar este objetivo general, se han definido los siguientes objetivos específicos:

1. Desarrollar un módulo de autenticación y seguridad para el registro e inicio de sesión de usuarios mediante tokens JWT, con soporte para perfiles diferenciados de inquilino y propietario.

2. Permitir la gestión y edición del perfil personal, incluyendo datos de contacto e información relevante para la convivencia.

3. Crear un sistema de publicación y exploración de anuncios de habitaciones con filtros por ubicación, precio, características del inmueble y normas de la casa.

4. Implementar un canal de mensajería contextual que permita a los interesados comunicarse con el propietario sobre un anuncio concreto sin obligar a formalizar una reserva previa.

5. Gestionar el ciclo de vida de las solicitudes de reserva de forma controlada, garantizando un periodo de gracia posterior a la estancia para la resolución de posibles incidencias o fianzas.

6. Diseñar un sistema de tareas domésticas gamificado, con asignación rotativa justa de turnos (Round-Robin), mecanismos de rescate puntual de tareas pendientes y una tabla de puntuaciones del piso.

7. Construir un módulo de auditoría del hogar que registre de forma transparente eventos clave de la convivencia, tales como la realización de tareas o la entrada y salida de compañeros.

8. Integrar un asistente inteligente basado en el estándar Model Context Protocol (MCP) para resolver dudas sobre normas, consultar turnos de limpieza o buscar habitaciones según el ambiente del piso, sin permitir modificaciones automáticas en la base de datos.

A continuación, se detalla la estructura técnica global del proyecto (ilustrada en la figura 1.1), describiendo las herramientas y tecnologías utilizadas en cada una de sus capas: cliente web, servidor principal, microservicio de inteligencia artificial y almacenamiento.

En la parte inferior de la arquitectura se sitúa el usuario final, quien interactúa con la plataforma a través del navegador web mediante una interfaz de usuario reactiva desarrollada con React, TypeScript y Tailwind CSS. El empaquetado y la compilación se gestionan con Vite para lograr tiempos de carga rápidos y un flujo de desarrollo ágil. Para la gestión del estado remoto y la sincronización con el servidor se emplea TanStack Query, aplicando una estrategia de consultas periódicas controladas (Smart Polling) que refresca los datos de mensajería y tareas solo mientras la pestaña permanece activa, reduciendo peticiones innecesarias sin requerir infraestructura adicional de sockets.

En el nivel intermedio se encuentra el backend principal, implementado en Java 21 utilizando el framework Spring Boot 3. Este servidor expone una API REST organizada por dominios y siguiendo principios de responsabilidad única (SOLID). Se encarga de procesar las reglas de negocio críticas: el cálculo determinista de las rotaciones de limpieza, la desvinculación reactiva de solicitudes mediante eventos de dominio internos (@EventListener) y la protección de accesos concurrentes mediante bloqueos a nivel de base de datos. Además, este mismo servicio actúa como orquestador cognitivo apoyándose en Spring AI, gestionando la comunicación con modelos de lenguaje a través de la API de Groq para responder a las consultas del asistente en formato estructurado.

Junto al servidor central se integra un microservicio independiente desarrollado en Node.js con TypeScript, cuyo único propósito es funcionar como servidor MCP oficial exponiendo eventos mediante Server-Sent Events (SSE). Este servicio declara y ejecuta las herramientas de consulta autorizadas (búsqueda avanzada de anuncios, estado de tareas y cola de reportes de moderación) propagando el token de seguridad del usuario en cada conexión, manteniendo una política estricta de solo lectura que impide cualquier mutación accidental de datos desde la inteligencia artificial.

Por último, en la capa superior de persistencia, se utiliza PostgreSQL como base de datos relacional principal. A través de JPA y Hibernate se gestiona el almacenamiento estructurado de usuarios, anuncios, hilos de conversación, históricos de auditoría y series de tareas recu-

rrentes, aplicando índices compuestos y restricciones de unicidad para evitar duplicidades y garantizar la integridad de la información en todo momento.

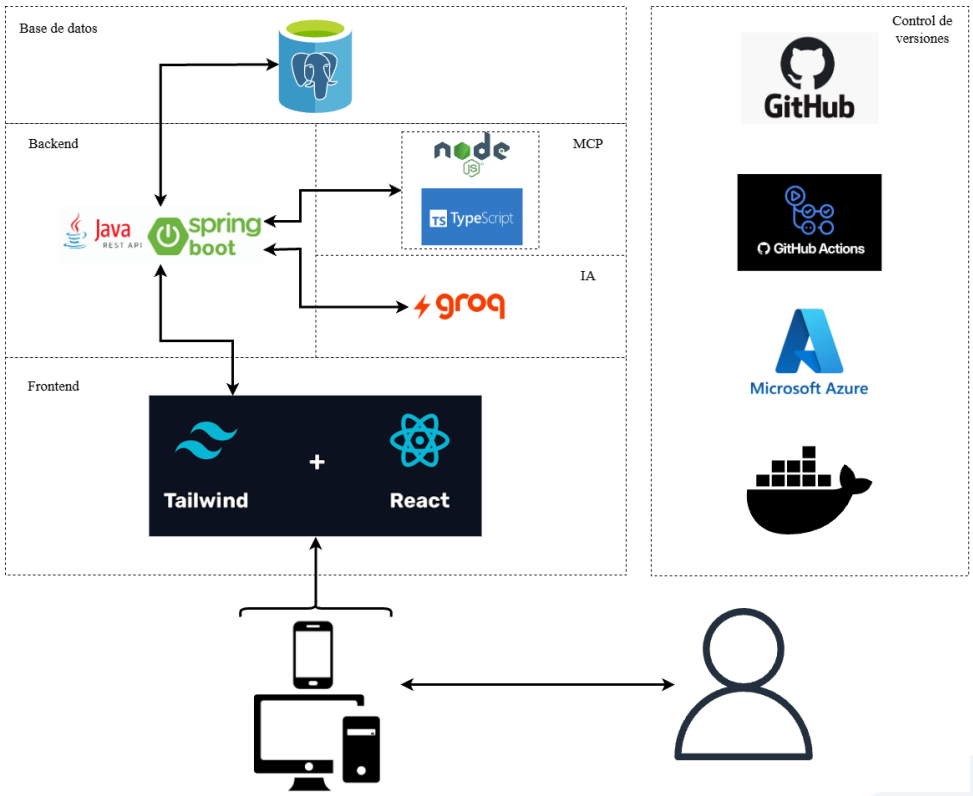


Figura 1.1: Estructura técnica global y arquitectura de la plataforma Colivi

1.3 PLANIFICACION

La planificación de este proyecto se estructuró siguiendo un ciclo de vida clásico de ingeniería del software, orientado a cubrir de manera rigurosa la carga de trabajo de 12 ECTS (300 horas lectivas) establecida para el Trabajo Fin de Grado. El cronograma contempla desde las etapas analíticas previas hasta la integración de servicios avanzados de inteligencia artificial, la adaptación multiplataforma y la redacción final de la memoria académica. A continuación, se presenta la visión general de la estimación inicial de tiempos, detallando la dedicación prevista para cada fase y la justificación de sus dependencias lógicas.

Antes de comenzar la construcción del sistema, se definieron tres etapas preliminares: el estudio de las herramientas tecnológicas (30 horas), el análisis de requisitos funcionales y no funcionales (30 horas) y la realización del diseño conceptual y prototipos de interfaz (40 horas). Estas actividades, que sumaron una dedicación conjunta de 100 horas a lo largo de las primeras cuatro semanas, resultaron críticas para evaluar la viabilidad de la arquitectura distribuida, familiarizarse con el estándar Model Context Protocol (MCP) y la API de inferencia de Groq, modelar el esquema relacional en PostgreSQL y concebir los flujos de interacción de usuario en mockups interactivos antes de codificar la solución.

Posteriormente, se planificó la etapa técnica central, que concentró la mayor parte del esfuerzo con un total de 170 horas de desarrollo. El núcleo principal correspondió a la implementación de la aplicación web y sus microservicios (110 horas), abarcando la lógica de negocio en Spring Boot 3, el frontend reactivo con React y TanStack Query, y el servidor MCP en Node.js. Como derivación directa de esta base funcional, se programó la adaptación ergonómica a aplicación móvil mediante Capacitor para la generación del ejecutable Android (25 horas), la configuración de la infraestructura de despliegue mediante contenedores Docker y servicios cloud en Microsoft Azure (20 horas), y una fase final de pruebas exhaustivas y control de calidad (15 horas) para validar la seguridad y robustez del sistema.

Por último, se reservó un bloque final de 30 horas de dedicación exclusiva para la redacción de la memoria académica en la plantilla oficial de la universidad, la elaboración de diagramas formales y la preparación de la defensa pública. Esta fase de documentación se programó con una dependencia estricta respecto al desarrollo técnico, asegurando que el documento recogiera con fidelidad el comportamiento final del sistema una vez verificado y estabilizado al 100

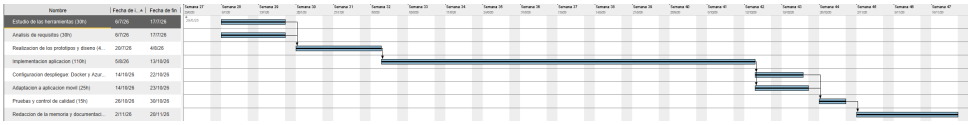


Figura 1.2: Planificación Temporal General

La fase principal del desarrollo se centró en la implementación de la aplicación, planificada con una dedicación de 110 horas. Para optimizar el tiempo de desarrollo, el trabajo se dividió entre tareas que se podían construir en paralelo y aquellas que requerían un orden secuencial estricto debido a dependencias directas en la base de datos y en la lógica de negocio.

En cuanto a las tareas abordadas de forma simultánea, se trabajó en el levantamiento de la infraestructura base del backend en Spring Boot y la definición de las tablas en PostgreSQL, mientras que en el frontend se maquetaba la estructura de navegación general con React y Tailwind. Del mismo modo, el desarrollo de los módulos de autenticación local con JWT y la integración de inicio de sesión con Google avanzaron a la par que sus respectivas vistas en el cliente (registro, inicio de sesión y recuperación de contraseña), ya que compartían la gestión de estados y validación de tokens. Más adelante, también se elaboraron en paralelo la interfaz del catálogo de alojamientos con sus filtros de búsqueda y el sistema de mapas interactivos, aprovechando que ambos componentes consumían los mismos endpoints de inmuebles disponibles.

Por otro lado, la mayor parte de las funcionalidades troncales tuvieron que implementarse de forma secuencial, siguiendo una cadena lógica de dependencias:

1. Gestión de alojamientos y catálogo: Antes de poder reservar, era imprescindible disponer del módulo de publicación de propiedades físicas y anuncios, permitiendo a los propietarios subir fotos y definir comodidades.

2. Solicitudes de reserva y pago de fianzas: Con el catálogo funcionando, se construyó el flujo de reservas con validación de meses completos, control de concurrencia y depósito de fianza.

3. Módulo de Hogares (Convivencia): Este apartado solo podía entrar en juego una vez confirmada y pagada la reserva. Actúa como el contenedor central del piso compartido donde conviven los inquilinos aceptados.

4. Gastos compartidos y tareas domésticas (Chores): Ambas funciones requerían obligatoriamente que el «Hogar» estuviese ya creado. Primero se implementó el reparto proporcional de facturas con cálculo dinámico de deudas y registro cronológico de actividad, y posteriormente el sistema de asignación rotativa de limpieza con puntuaciones y tareas recurrentes.

5. Mensajería interna: Se desarrolló tras tener definida la convivencia, permitiendo la comunicación directa entre los compañeros de un mismo hogar o con el propietario del inmueble.

6. Asistente virtual (Spring AI y MCP): Fue de los últimos bloques técnicos en integrarse. No tenía sentido construir las herramientas de consulta del servidor MCP (Model Context Protocol) ni conectar el chat sin tener antes estabilizadas y probadas las APIs internas de búsqueda de catálogo y de estado de tareas domésticas, de las cuales extrae los datos.

7. Moderación administrativa, reseñas y RGPD: Finalmente, se implementó el canal de denuncias con resolución administrativa en cascada, las valoraciones entre usuarios y la capa de consentimiento legal para cerrar el sistema antes de pasar a la fase móvil y de despliegue.

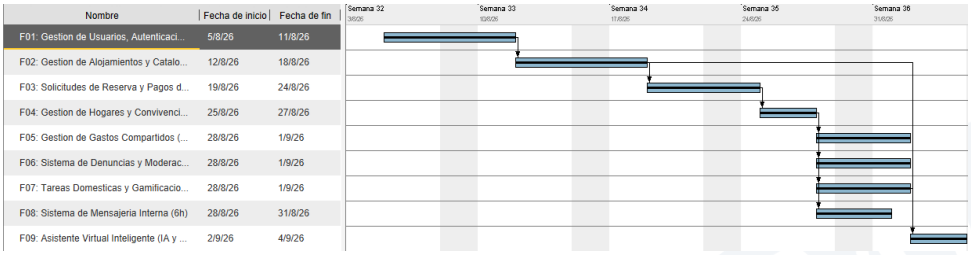


Figura 1.3: Planificación Temporal de la implementación

1.4 Apps Similares

Antes de abordar el desarrollo del sistema, se llevó a cabo un estudio de las principales soluciones existentes en el mercado que guardan relación con los servicios que plantea este proyecto. El propósito de este análisis comparativo fue doble: por un lado, identificar patrones consolidados de diseño e interacción en las fases de búsqueda y convivencia; por otro, evidenciar las carencias funcionales del software actual y justificar la necesidad de una plataforma integral como Colivi.

Dentro del conjunto de herramientas evaluadas se seleccionaron portales de alquiler tradicionales como Idealista [1], plataformas de alojamiento temporal y reservas como Airbnb [2], aplicaciones especializadas en alquiler de habitaciones y búsqueda de compañeros como Badi [3], herramientas de gestión contable compartida como Tricount [4], y aplicaciones orientadas a la organización doméstica como Flatastic [5].

En primer lugar, Idealista [1] (figura 1.4a) representa el referente indiscutible en el mercado inmobiliario español para la publicación y búsqueda de inmuebles. Cuenta con una base

de datos masiva, un motor de filtros geográficos muy optimizado y herramientas de contacto directo con anunciantes. Sin embargo, su modelo de negocio está concebido para el alquiler tradicional de viviendas completas o habitaciones desde una perspectiva puramente transaccional. Una vez que el contrato se formaliza, la plataforma desaparece de la ecuación: no ofrece mecanismos de depósito seguro de fianza dentro de la aplicación, carece de funciones para gestionar la convivencia y no contempla ningún tipo de comunidad o afinidad entre inquilinos.

En segundo lugar, Airbnb [2] (figura 1.4b) destaca por su robusto sistema de reservas, pasarela de pagos integrada, gestión de garantías y verificación de identidades mediante reseñas mutuas. No obstante, su orientación es casi exclusivamente turística o de estancias cortas. Aunque permite estancias de varias semanas, sus comisiones por servicio no son viables para alquileres residenciales o de media estancia. Además, no contempla la figura del piso compartido entre desconocidos que necesitan coordinar su día a día, delegando cualquier acuerdo de convivencia a la comunicación externa entre huéspedes y anfitriones.

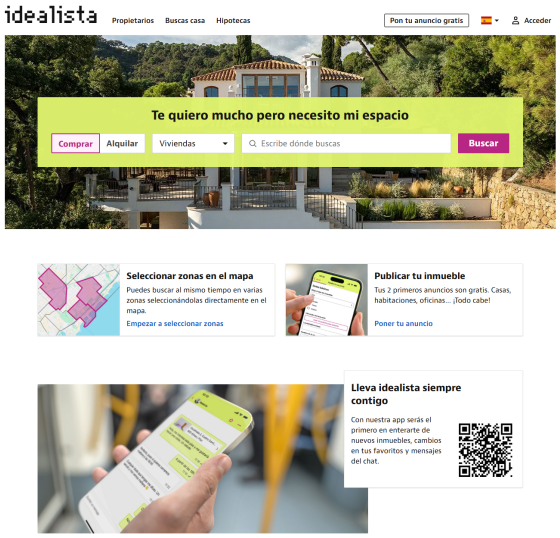
Por otro lado, Badi [3] (figura 1.4c) fue concebida de forma específica para solucionar el problema del alquiler por habitaciones en grandes urbes, permitiendo buscar tanto cuartos disponibles como compañeros de piso según perfiles personales y compatibilidad básica mediante un chat interno. Pese a ser la propuesta más cercana a la fase inicial de captación de Colivi, su alcance queda truncado en el momento de la firma: no dispone de un módulo de gestión del hogar que acompañe a los usuarios durante los meses de convivencia. Toda la organización interna del piso (cuentas, suministros, mantenimiento) queda fuera de su alcance.

En lo relativo a la economía del hogar, aplicaciones como Tricount [4] (figura 1.4d) son ampliamente utilizadas por estudiantes y jóvenes profesionales para el reparto de facturas comunes. Destacan por su capacidad para simplificar grafos de deudas cruzadas y registrar pagos entre múltiples participantes. Pese a su efectividad matemática, son aplicaciones aisladas: no tienen noción del inmueble físico, no verifican la legitimidad de los gastos con el contrato de alquiler ni permiten liquidar fianzas contractuales de forma segura, exigiendo que los inquilinos introduzcan los datos manualmente sin ningún vínculo con su arrendador.

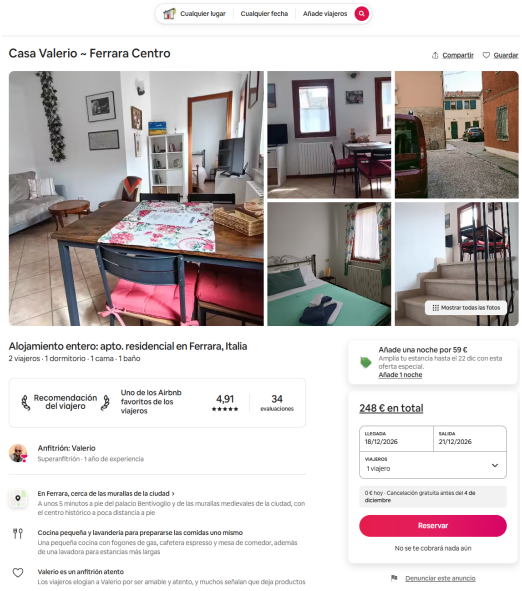
En conclusión, el panorama tecnológico actual se encuentra fragmentado en silos funcionales. Ninguna de las alternativas evaluadas cubre el ciclo integral del coliving: desde la búsqueda y validación inicial del alojamiento con fianza segura, hasta la gestión activa de la convivencia (gastos, turnos rotativos y mensajería), pasando por la integración de un asistente conversacional inteligente capaz de interactuar con el estado operativo del piso. Colivi nace para unificar estas facetas bajo un único ecosistema web y móvil.

Tabla 1.1: Comparativa de características entre plataformas analizadas y Colivi

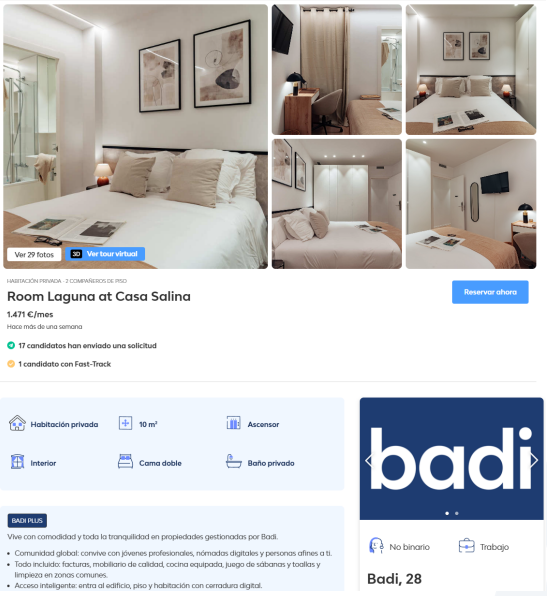
Plataforma	Catálogo y Búsqueda	Reserva y Fianza Integrada	Reparto de Gastos Comunes	Gamificación de Tareas	Asistente IA (MCP)	Enfoque Coliving
Idealista	Sí	No	No	No	No	No
Airbnb	Sí	Sí	No	No	No	No
Badi	Sí	No	No	No	No	Sí
Tricount	No	No	Sí	No	No	No
Colivi (Propuesta)	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**



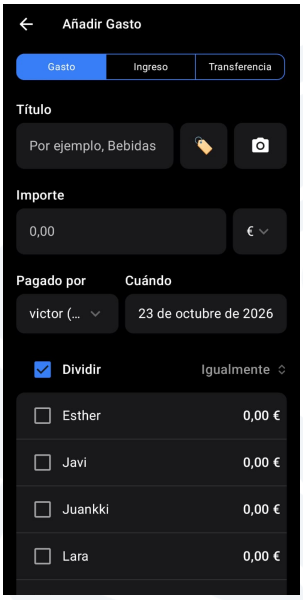
(a) Búsqueda en Idealista



(b) Detalles y reserva en Airbnb



(c) Habitaciones y perfiles en Badi



(d) Reparto de gastos en Tricount

Figura 1.4: Interfaces de las principales soluciones analizadas en el mercado

Para interpretar de manera objetiva la tabla comparativa, conviene delimitar los criterios asignados a cada una de las columnas evaluadas:

- **Catálogo y Búsqueda**: Indica si la plataforma dispone de un motor público de exploración con filtros geográficos y características del inmueble (precio, comodidades, disponibilidad de habitaciones). Portales como Idealista, Airbnb y Badi satisfacen este requisito con solvencia, mientras que utilidades como Tricount o Flatastic carecen por definición de esta vertiente comercial.

- **Reserva y Fianza Integrada**: Hace referencia a la capacidad de formalizar contractualmente la estancia y retener o transferir depósitos de fianza mediante pasarelas auditables dentro del propio sistema. Solo Airbnb cumple este punto en el mercado actual, ya que Idealista y Badi delegan el pago y la garantía a acuerdos externos privados entre las partes.

- **Reparto de Gastos Comunes**: Evalúa si el sistema ofrece un motor de balances para dividir facturas, calcular saldos individuales y registrar liquidaciones. Es el núcleo de Tricount y una función presente en Flatastic, pero inexistente en las plataformas inmobiliarias.

- **Gamificación de Tareas**: Se exige que la herramienta incluya mecanismos dinámicos de distribución del trabajo doméstico (como asignaciones rotativas periódicas y métricas o tableros de puntuación). Solo Flatastic implementa una aproximación a este concepto, aunque de manera básica y sin auditoría cruzada con el resto del hogar.

- **Asistente IA (MCP)**: Evalúa la existencia de un agente conversacional contextualizado mediante protocolos como Model Context Protocol, capaz no solo de responder consultas generales, sino de consultar el estado de la base de datos (por ejemplo, comprobar qué gastos siguen pendientes o listar las tareas asignadas para la semana en curso). Ninguna de las herramientas existentes ofrece integración con modelos de lenguaje orientados a la gestión operativa del inmueble.

- **Enfoque Coliving**: Determina si el diseño de la aplicación está optimizado pensando en el modelo de habitación privada con áreas compartidas y convivencia grupal entre pares. Badi y Flatastic reciben esta marca por su enfoque específico en compañeros de piso, a diferencia de Idealista (enfocada al mercado residencial genérico) o Airbnb (diseñada para el hospedaje turístico).

En algunos casos, la asignación de estas características puede presentar matices. Por ejemplo, en Badi se gestiona la reserva inicial de la habitación a través de un depósito temporal de reserva, pero no ofrece la custodia ni la gestión contable de la fianza del contrato de arrendamiento durante la vigencia del alquiler, motivo por el cual no se considera una solución de fianza integrada completa. De igual manera, aunque plataformas como Airbnb han comenzado a experimentar con algoritmos de atención al cliente automatizados, no disponen de agentes conversacionales capaces de consultar e interactuar con la lógica interna de convivencia de los residentes.

1.5 CONTENIDO DE LA MEMORIA

La presente memoria se encuentra estructurada en diversos capítulos que recogen, de manera sistemática y progresiva, las fases de análisis, diseño, desarrollo y validación de Colivi, una plataforma web y móvil asistida por inteligencia artificial orientada a la gestión integral del alquiler en régimen de coliving y a la convivencia entre residentes. A continuación, se resume de forma sucinta el propósito y contenido de cada una de las partes que integran este trabajo:

- Introducción: se contextualiza la creciente demanda del modelo residencial de coliving entre estudiantes y jóvenes profesionales, exponiendo las problemáticas actuales asociadas a la dispersión de herramientas digitales durante la búsqueda de piso y la convivencia diaria.

- Justificación: se detallan las razones técnicas y sociales que motivan la elección de esta propuesta, destacando la oportunidad de resolver la fragmentación operativa del alquiler compartido unificando el ciclo completo en una sola solución con soporte inteligente.

- Objetivos: se formula el objetivo general del proyecto y se desglosan los objetivos específicos, delimitando el alcance funcional de la plataforma tanto en la vertiente web como en su empaquetado móvil.

- Planificación: se describe la organización temporal del desarrollo a partir de un ciclo de vida clásico de ingeniería del software, exponiendo las fases del proyecto, el cronograma de trabajo planificado en 300 horas lectivas y la trazabilidad de las dependencias lógicas.

- Aplicaciones similares: se ofrece un análisis comparativo frente a soluciones existentes en el mercado (portales de búsqueda inmobiliaria, plataformas de alojamiento temporal y utilidades de reparto de gastos), delimitando los criterios funcionales que diferencian y fundamentan la propuesta de Colivi.

- Herramientas y tecnologías: se justifican las decisiones metodológicas y la selección del stack tecnológico empleado, profundizando en el backend con Java 21 y Spring Boot 3, el frontend reactivo con React, TypeScript y Tailwind CSS, la persistencia relacional con PostgreSQL, la integración del protocolo Model Context Protocol (MCP) con Spring AI y Groq, y la adaptación móvil mediante Capacitor.

- Análisis y diseño del sistema: se documentan los diferentes roles de usuario, el catálogo riguroso de requisitos funcionales y no funcionales, los diagramas de casos de uso, el modelo entidad-relación de la base de datos y la arquitectura distribuida del sistema desacoplada bajo principios SOLID.

- Implementación del sistema: se expone detalladamente el desarrollo técnico modular de la plataforma, abordando el control de acceso sin estado con JWT, el catálogo con mapas interactivos, el flujo de reservas con fianza integrada y control de concurrencia, los módulos de convivencia del hogar (gastos compartidos, tareas domésticas con asignación rotativa y mensajería interna) y el microservicio MCP que da soporte al asistente conversacional.

- Adaptabilidad móvil y despliegue: se describe la optimización ergonómica de la interfaz de usuario para dispositivos móviles, el empaquetado del aplicativo para Android y la configuración de la infraestructura mediante contenedores Docker y servicios cloud en Microsoft Azure con automatización CI/CD.

- Plan de pruebas y aseguramiento de la calidad: se detalla la estrategia de verificación adoptada, cubriendo las pruebas unitarias y de integración de la lógica de negocio, las comprobaciones de los endpoints seguros y el análisis de incidencias técnicas resueltas durante el desarrollo.

- Estudio de costes: se desglosa el presupuesto global estimado para la ejecución del proyecto, cuantificando los costes derivados del esfuerzo humano (horas de ingeniería), la infraestructura cloud, las herramientas de desarrollo y los recursos materiales requeridos.

- Conclusiones y líneas futuras: se sintetizan las metas alcanzadas y el grado de cumplimiento de los objetivos planteados, proponiendo posibles vías de evolución técnica para extender las capacidades operativas y de inteligencia del sistema.

- Bibliografía: se recopilan formalmente todas las referencias bibliográficas, documentación técnica oficial, especificaciones de protocolos, estándares y fuentes académicas consultadas a lo largo de la investigación y construcción de la memoria.

2 Herramientas y metodologías

Durante el desarrollo de este proyecto se consideró fundamental adoptar una metodología que combinara el control riguroso de la calidad del código con la flexibilidad necesaria para abordar una arquitectura distribuida. Al tratarse de un trabajo individual con un grado notable de interconexión entre componentes, se optó por un enfoque ágil adaptado que combinó entregas iterativas con la práctica de desarrollo guiado por pruebas o TDD (*Test-Driven Development*). Esta estrategia facilitó estructurar el trabajo en ciclos cortos de especificación, codificación y validación, asegurando la estabilidad de los módulos antes de introducir nuevas dependencias.

El uso de TDD resultó especialmente relevante en los componentes con lógica de negocio crítica, como el motor contable de liquidación de gastos comunes y la gestión de concurrencia en las reservas. La escritura previa de las pruebas unitarias e integradas permitió fijar el comportamiento de las entidades y servicios con precisión, reduciendo sustancialmente la aparición de errores en fases avanzadas. De forma complementaria, este flujo de trabajo se apoyó en herramientas de inteligencia artificial generativa, utilizándolas como un asistente técnico para agilizar la creación de baterías de pruebas, evaluar casos límite y orientar refactorizaciones de código enfocadas al cumplimiento estricto de los principios SOLID (*Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion*) y al desacoplamiento entre capas.

Para la construcción de la plataforma se seleccionó un conjunto de tecnologías y herramientas orientado a garantizar la mantenibilidad, el tipado estricto y la escalabilidad del sistema:

- Entorno de desarrollo: el proyecto se articuló mediante el uso de Antigravity como entorno principal de trabajo, facilitando la navegación coordinada entre los distintos repositorios, la depuración y la automatización de tareas.
- Lógica de negocio y servicios: el backend principal se implementó con Java 21 sobre el marco de trabajo Spring Boot 3, incorporando Spring AI para la conexión con modelos de lenguaje y la gestión de llamadas a funciones. De forma independiente se construyó un servidor para el protocolo MCP (*Model Context Protocol*) desarrollado con Node.js y TypeScript, comunicándose con el núcleo a través de eventos enviados por el servidor o SSE (*Server-Sent Events*).
- Interfaz web y empaquetado móvil: la aplicación web se desarrolló con React, utilizando Vite como herramienta de construcción y TypeScript para asegurar el tipado estático del cliente. Para el diseño visual se empleó Tailwind CSS, mientras que la sincronización de

las peticiones asíncronas se gestionó mediante TanStack Query. La adaptación hacia dispositivos móviles se resolvió mediante Capacitor, permitiendo empaquetar la base del cliente web directamente para la plataforma Android.

- Almacenamiento de datos: la persistencia relacional se confió al sistema gestor de bases de datos PostgreSQL, administrando las transacciones y entidades mediante JPA (*Java Persistence API*) e Hibernate.
- Prototipado y diseño de interfaz: la experiencia de usuario y los esquemas preliminares de navegación se diseñaron en Figma, elaborando maquetas interactivas tanto para la vista de escritorio como para resoluciones móviles.
- Diagramación formal: los diagramas de casos de uso y esquemas de comportamiento se generaron mediante PlantUML a partir de código fuente, complementados con diagramas de bloques elaborados en draw.io para representar la arquitectura global.
- Control de versiones y cronograma: la gestión del código fuente se realizó en la plataforma GitHub, aplicando un flujo basado en ramas y solicitudes de extracción (*pull requests*) para auditar cada incremento de funcionalidad. El control temporal y la verificación de dependencias entre fases se llevaron a cabo mediante la herramienta GanttProject.

A continuación, se describen con detalle las decisiones metodológicas adoptadas, así como la configuración técnica y el rol específico de cada una de las herramientas a lo largo de las distintas etapas del proyecto.

2.1 METODOLOGÍAS USADAS

Según la guía del cuerpo de conocimiento de la ingeniería del software o SWEBOK (*Software Engineering Body of Knowledge*) [6], los procesos de desarrollo deben estructurarse en etapas sistemáticas que permitan proyectar, supervisar y reajustar el avance en función de las restricciones operativas y los objetivos establecidos. Esta concepción metodológica abarca desde los modelos predictivos secuenciales hasta las estrategias adaptativas fundamentadas en marcos ágiles. La elección de una metodología condiciona de forma directa la trazabilidad del código, la capacidad de respuesta ante requisitos emergentes y el aseguramiento de la calidad técnica. Con el fin de fundamentar la estrategia de trabajo de Colivi, a continuación se analizan los paradigmas más extendidos en la ingeniería del software contemporánea.

2.1.1 Revisión de enfoques metodológicos

Por un lado, el modelo tradicional en cascada plantea una progresión estrictamente secuencial a través de fases cerradas (análisis de requisitos, diseño de la arquitectura, codificación, verificación y mantenimiento). Cada etapa exige la validación completa de sus entregables para iniciar la siguiente. Si bien aporta certidumbre contractual en entornos donde la especificación funcional está congelada desde el inicio, resulta excesivamente rígido en

sistemas con integración de servicios desacoplados o dependencias experimentales, donde cualquier cambio arquitectónico tardío acarrea un sobrecoste elevado.

Por otro lado, marcos ágiles como Scrum plantean ciclos iterativos de corta duración denominados *sprints*, orientados a la entrega de incrementos funcionales utilizables en intervalos de dos a cuatro semanas. Scrum define ceremonias específicas (reuniones diarias de sincronización, planificación de iteración, retrospectivas) y distribuye responsabilidades entre roles formales, tales como el facilitador del flujo de trabajo o *Scrum Master* y el responsable del producto o *Product Owner*. Aunque constituye el estándar predominante en equipos de desarrollo corporativos, la sobrecarga organizativa de sus ceremonias carece de aplicabilidad práctica en un contexto de desarrollo unipersonal. En la figura 2.1 se ilustra una comparativa gráfica representativa entre los resultados históricos de proyectos desarrollados bajo cascada frente a enfoques ágiles.

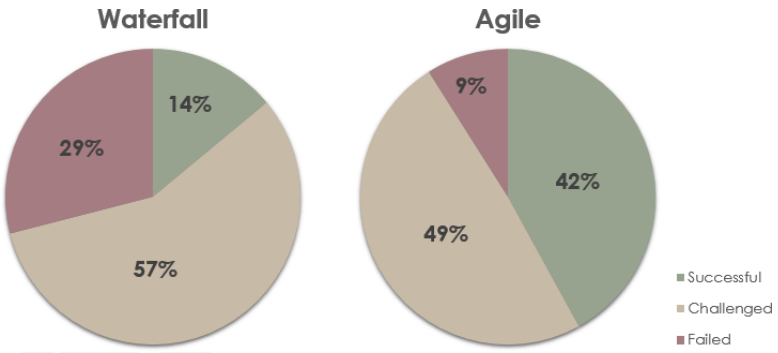


Figura 2.1: Diferencia de éxito entre metodologías ágiles y tradicionales en cascada

Una alternativa complementaria es Kanban, un método centrado en la visualización continua del flujo de trabajo y en el control del trabajo en curso o WIP (*Work In Progress*). Mediante tableros estructurados en estados secuenciales (como «Pendiente», «En curso» y «Completado»), como el reflejado en la figura 2.2, Kanban permite gestionar la evolución de las tareas sin imponer cadencias temporales fijas. Esta flexibilidad facilita priorizar el trabajo según las dependencias técnicas reales, optimizando el rendimiento individual y la detección temprana de cuellos de botella.

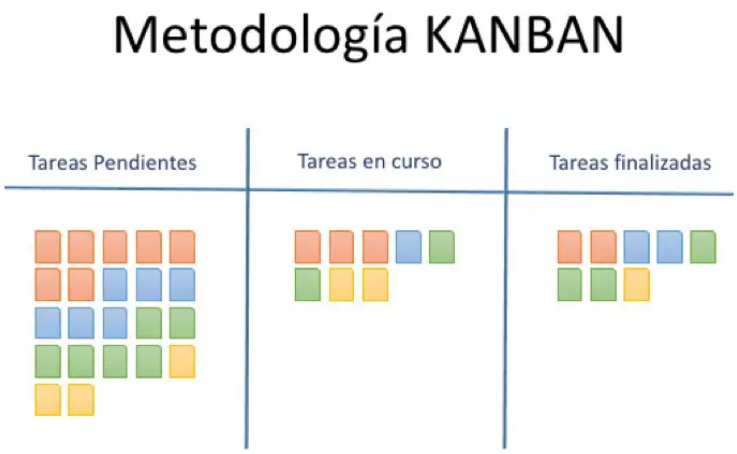


Figura 2.2: Tablero Kanban representativo del flujo de desarrollo ágil

Por último, la metodología de programación extrema o XP (*Extreme Programming*) sitúa el foco en la excelencia técnica y la mitigación del riesgo mediante prácticas disciplinadas de codificación. Entre sus fundamentos destacan la integración continua del código, la refactorización sistemática orientada a la simplicidad y el desarrollo guiado por pruebas o TDD (*Test-Driven Development*), donde los casos de prueba automatizados se redactan antes de la lógica de producción.

2.1.2 Análisis comparativo y selección para el proyecto

La naturaleza técnica de Colivi —caracterizada por un backend modular en Spring Boot 3, un microservicio independiente bajo el protocolo MCP (*Model Context Protocol*), una interfaz reactiva en React y el empaquetado móvil con Capacitor— desaconsejaba el empleo de una cascada pura. La integración de modelos de lenguaje mediante inferencia remota exigía admitir ajustes de diseño a medida que se validaban las herramientas de contexto del servidor.

Sin embargo, adoptar Scrum de manera dogmática resultaba inviable ante la inexistencia de un equipo multidisciplinar con el que coordinar reuniones diarias o repartir roles de gestión. Por tanto, para este Trabajo Fin de Grado se optó por un enfoque metodológico híbrido:

- Gestión visual del flujo de trabajo: se adoptó la base operativa de Kanban para organizar las tareas en función de sus estados y dependencias lógicas, utilizando el soporte de gestión de incidencias y tableros integrado en GitHub Projects.

- Disciplina de ingeniería técnica: se incorporaron prácticas troncales de XP, haciendo especial énfasis en el desarrollo guiado por pruebas (TDD), cuyo ciclo iterativo de diseño, prueba y refactorización se sintetiza en la figura 2.3. Esta práctica permitió blindar los módulos con alta complejidad algorítmica y financiera (como el reparto proporcional de gastos o el control de concurrencia en reservas), manteniendo una refactorización orientada al cumplimiento de los principios SOLID (*Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion*).

- Integración continua asistida: el ciclo de desarrollo se apoyó en el entorno Antigravity y en herramientas de inteligencia artificial para agilizar la cobertura de casos límite en pruebas unitarias con JUnit y Mockito, complementando la verificación manual con flujos automatizados de compilación y pruebas en ramas de características (*feature branches*).

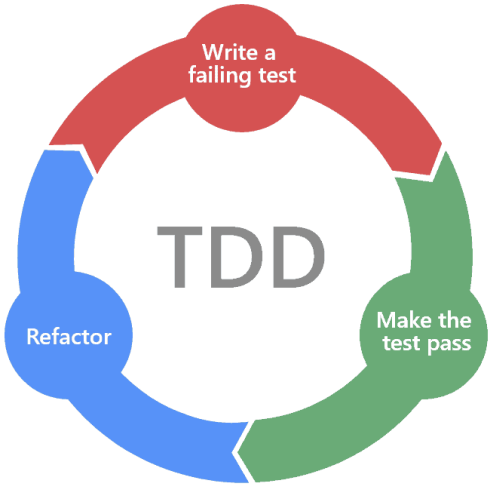


Figura 2.3: Ciclo iterativo de la metodología TDD (*Test-Driven Development*)

2.2 Herramientas usadas

En esta sección se describen los entornos, lenguajes, marcos de trabajo y utilidades auxiliares que se utilizaron a lo largo del ciclo de vida del proyecto, justificando técnicamente la elección de cada componente frente a otras alternativas disponibles.

2.2.1 Entornos de desarrollo y asistencia técnica

Para la construcción del sistema se seleccionaron entornos de trabajo específicos en función de las demandas de cada capa de la arquitectura (figura 2.4). En el caso del backend y la orquestación general de servicios, se utilizó Antigravity como entorno de trabajo integrado. Esta herramienta proporcionó una gestión eficiente de los espacios de trabajo sobre los repositorios del proyecto, combinando terminales concurrentes, ejecución de contenedores locales y soporte nativo para flujos de codificación asistida. Frente a entornos tradicionales como Eclipse o NetBeans, Antigravity ofreció un consumo de recursos sensiblemente menor y una navegación ágil entre el código del servidor en Spring Boot y el microservicio auxiliar en Node.js.

Para el desarrollo del frontend web y la capa móvil se optó por Visual Studio Code. Su ecosistema de extensiones, especialmente las relativas al análisis estático de TypeScript, el formateo con Prettier y la integración de ramas con Git, permitió mantener un ritmo de desarrollo uniforme. Aunque se evaluó la posibilidad de unificar todo el trabajo bajo un único entorno pesado como IntelliJ IDEA Ultimate, se prefirió la combinación de Antigravity y Visual Studio Code por su ligereza y facilidad de adaptación al cambio constante de contexto entre lenguajes.

Por último, en lo relativo al soporte móvil, se recurrió a Android Studio de manera puntual. Al basarse la vertiente móvil en una aproximación híbrida mediante Capacitor, el grueso de la lógica se implementó en la base web; no obstante, Android Studio resultó indispensable para compilar el código nativo, generar la clave de firmado del paquete de aplicación Android o APK (*Android Package Kit*), depurar fallos en el puente de comunicación mediante el inspector web de Android y realizar pruebas sobre emuladores oficiales con distintas versiones del sistema operativo.

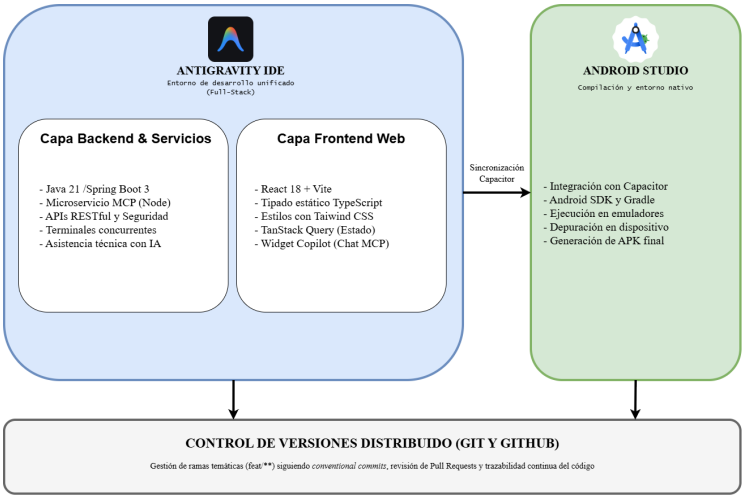


Figura 2.4: Entornos de desarrollo utilizados en el proyecto. Elaboración propia

2.2.2 Lenguajes, marcos de trabajo y servicios

La arquitectura distribuida de Colivi requirió una selección tecnológica coherente entre sus diferentes módulos operativos. Para el backend se eligió Java 21 junto con el marco de trabajo Spring Boot 3. La versión 21 de Java aportó características de lenguaje consolidadas como los registros (*records*) y la concordancia de patrones (*pattern matching*), lo que redujo de forma sustancial la redundancia en los objetos de transferencia de datos o DTO (*Data Transfer Object*). Por su parte, Spring Boot 3 ofreció una base robusta para la gestión de inyección de dependencias, transacciones relacionales y seguridad perimetral sin estado mediante Spring Security y filtros de autenticación basados en tókenes web de JSON o JWT (*JSON Web Tokens*). Frente a alternativas como Django en Python o Express en Node.js, Spring Boot garantizó un tipado estricto en la lógica de negocio contable y facilitó la integración directa con Spring AI para interactuar con modelos de lenguaje remotos a través de la interfaz de programación de aplicaciones o API (*Application Programming Interface*) de Groq. En el microservicio del asistente inteligente se optó por Node.js con TypeScript, implementando la especificación del protocolo de contexto de modelo o MCP (*Model Context Protocol*). Se decidió desacoplar este módulo del backend principal para permitir que el servidor de herramientas operara de manera independiente, comunicándose mediante transporte por eventos enviados por el servidor o SSE (*Server-Sent Events*). TypeScript proporcionó la comprobación estática de tipos necesaria para garantizar que los esquemas de entrada y salida de las herramientas del asistente concordaran con los contratos esperados por el orquestador.

En el desarrollo de la interfaz web se seleccionó React en su versión 18 sobre la herramienta de empaquetado Vite, escribiendo el código íntegramente en TypeScript. A diferencia de JavaScript plano, el uso de TypeScript evitó inconsistencias comunes en tiempo de ejecución al interactuar con las respuestas del servidor. Vite se prefirió frente a Create React App debido a su velocidad de compilación mediante módulos de ECMAScript (*ES Modules*) y al reemplazo en caliente de módulos o HMR (*Hot Module Replacement*). Para el diseño visual se adoptó Tailwind CSS, aprovechando su enfoque de utilidades para estructurar interfaces adaptables sin abandonar los archivos de componentes. La sincronización del estado del servidor se gestionó con TanStack Query, resolviendo de manera transparente el almace-

namiento en caché, la invalidación de consultas y la actualización en segundo plano de datos dinámicos como los balances de gastos o las alertas del hogar. Para la aplicación móvil se utilizó Capacitor como puente de integración sobre la base de React. En lugar de desarrollar una aplicación móvil desde cero en Kotlin o emplear marcos de trabajo separados como React Native, Capacitor permitió reutilizar más del 90 % del código del cliente web, envolviendo la interfaz en una vista web nativa con acceso directo a las características del dispositivo. Esta decisión optimizó el tiempo de desarrollo, asegurando una experiencia idéntica entre la versión de navegador y la aplicación instalada en el teléfono.

2.2.3 Almacenamiento y persistencia de datos

La persistencia del sistema se sustentó en el sistema gestor de bases de datos relacionales PostgreSQL en su versión 16 (figura 2.5). La naturaleza transaccional de Colivi —donde coexisten balances contables de facturas, reservas de habitaciones con control de solapamiento de fechas y auditorías de actividad— exigía el cumplimiento íntegro de las propiedades de atomicidad, consistencia, aislamiento y durabilidad o ACID (*Atomicity, Consistency, Isolation, Durability*). PostgreSQL demostró un comportamiento idóneo en el manejo de restricciones de integridad referencial complejas y consultas analíticas frente a motores no relacionales como MongoDB, los cuales se descartaron por carecer de mecanismos nativos estrictos para la resolución de saldos monetarios compartidos. La interacción entre la base de datos y la capa de dominio en Java se canalizó mediante la especificación JPA y el motor Hibernate, definiendo entidades bien mapeadas y repositorios con consultas específicas para mitigar problemas comunes de rendimiento como el cuello de botella de consultas secundarias sucesivas o problema de $N + 1$.



Figura 2.5: Sistema gestor de bases de datos PostgreSQL

2.2.4 Control de versiones, infraestructura y utilidades auxiliares

Para la gestión del código fuente se empleó Git como sistema de control de versiones distribuido, alojando los repositorios centrales en la plataforma GitHub. El progreso técnico

se organizó siguiendo un flujo de trabajo riguroso basado en ramas y solicitudes de incorporación de cambios o PR (*Pull Requests*). Esta estrategia permitió auditar cada funcionalidad antes de consolidarla en la rama de desarrollo, manteniendo un historial limpio de cambios y vinculando de forma unívoca cada confirmación con las tareas planificadas.

La contenedorización de los servicios locales se resolvió mediante Docker y Docker Compose. Se definieron archivos de configuración para desplegar simultáneamente la base de datos PostgreSQL, el backend en Spring Boot y el servidor MCP, garantizando que el entorno de ejecución fuera reproducible tanto en equipos de desarrollo como en las fases de despliegue sobre Microsoft Azure.

Para la fase de diseño visual se utilizó Figma (figura 2.6a) para maquetar la guía de estilos, los esquemas alámbricos y los prototipos de pantalla en resolución de escritorio y móvil, estableciendo la disposición de las tarjetas de alojamiento y los tableros del hogar antes de iniciar la maquetación.

Para el modelado formal de la arquitectura y el comportamiento del sistema se empleó PlantUML (figura 2.6b), lo que permitió versionar los diagramas de casos de uso y de secuencia como código fuente dentro del propio repositorio de documentación. Aquellos diagramas de bloques generales de alto nivel se confeccionaron con draw.io por su rapidez para exportar composiciones visuales limpias.

Finalmente, la verificación de los puntos de acceso de la API y las pruebas de comunicación de eventos SSE se realizaron mediante Postman (figura 2.6c) y cURL, comprobando manualmente los encabezados de autenticación JWT y los códigos de estado HTTP antes de proceder a la automatización de las baterías de pruebas unitarias e integradas con JUnit 5 y Mockito.



(a) Figma **(b)** PlantUML **(c)** Postman

Figura 2.6: Herramientas auxiliares empleadas para diseño de interfaz, modelado y verificación de API

3 Apartados del CUERPO DEL DOCUMENTO

El llamado “cuerpo del documento” es el grupo de apartados donde se describe como se ha desarrollado el trabajo. En el dominio de la Informática, un TFG debe mantener un equilibrio entre la componente de investigación y la componente puramente técnica asociada con el proyecto de desarrollo de software. Un projecto software conlleva la elaboración de una serie de memorias, cada una recopilando lo hecho a distintos niveles, ver figura 3.2. Sin embargo la memoria de un TFG es de carácter académico y ademas de artefactos, presupuestos y modelos debe incluir determinadas cuestiones que muestren como se han desarrollado las ideas propias. El punto de partida es la estructura típica de un articulo que se muestra en la figura 3.1.

La propuesta de estructura se define a continuación indicando que se incluiría en cada apartado. Lo importante en este caso es QUE HAY QUE INCLUIR la estructura del TFG final de cada alumno puede ser diferente, uniendo o separando los contenidos que se detallan a continuación.

Objetivos o cuestiones de investigación se establecerá el objetivo general y los específicos, es decir, cuestiones que serán respondidas con el trabajo que se va a realizar. No es imprescindible elaborar un apartado específico denominado OBJETIVOS, pues éstos puede quedar reflejados al final de la introducción

Planificación temporal Se debe incluir una referencia a como se han distribuido las tareas a lo largo del tiempo empleado en el TFG. Esta cuestión deriva de los proyectos técnicos de disciplinas de ingeniería. Puede representarse en forma de tabla o gráfico 3.3 y puede aparecer dentro del apartado de metodología.

Contextualización o estado del arte Se hará mención a los elementos conceptuales que sirven de base para la investigación, estudios previos relacionados con el problema planteado, etc. En este apartado es importante hacer un estudio de los trabajos previos o revisión bibliográfica [**? ?**]. Si como es habitual se tiene que utilizar una o varias herramientas software es en este apartado donde se recogen las alternativas consideradas y cual de ellas ha sido elegida y porqué.



Figura 3.1: Estructura clásica de un articulo de investigación



Figura 3.2: Estructura de documentos de un proyecto técnico

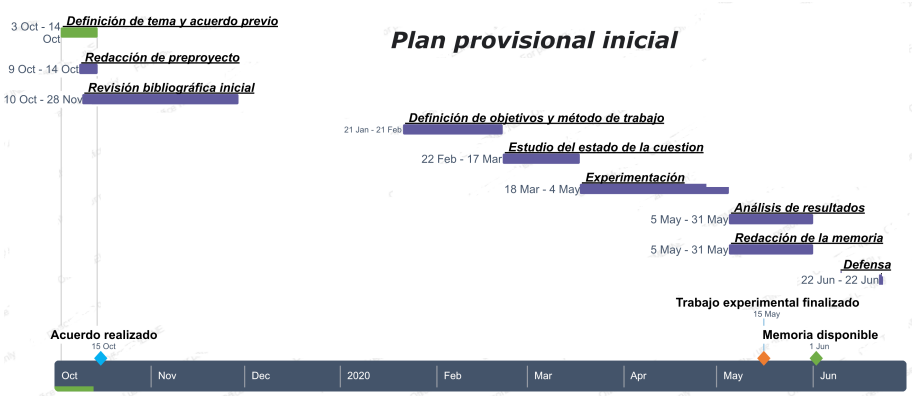


Figura 3.3: Planificación del proyecto

Metodología ¿Qué procedimiento se ha seguido para alcanzar los objetivos planteados? Se indicará el tipo o tipos de investigación, las técnicas y los procedimientos que serán utilizados para llevarla a cabo; se identificará la población y el tamaño de la muestra así como las técnicas e instrumentos de recolección de datos, si te trata de un trabajo empírico, la metodología de desarrollo de software seleccionada. O cualquier otra información sobre el proceso seguido. La planificación y objetivos también pueden estar en este apartado

Resultados Incluirá los resultados de la investigación o trabajo, así como el análisis y la discusión de los mismos, aunque algunos tutores prefieren dos apartados separados. En el caso de tratarse de un desarrollo de software en este apartado se incluirá la descripción del producto resultante y los modelos o artefactos softtware necesarios para su descripción. Pero el manual o tutorial detallado y los modelos completos deberían llevarse a los anexos.

La discusión conlleva revisar que aporta el trabajo realizado y se han conseguido los objetivos planteados inicialmente. Caso de tener una evaluación por parte de los clientes potenciales también aparecerá en este apartado.

4 Apartado BIBLIOGRAFIA

Este apartado es de vital importancia para tener éxito en la consecución del trabajo fin de grado. Los tribunales suelen prestar gran atención a este apartado, probablemente por su formación académica.

Es todo un mundo el de los estilos bibliográficos y la realización de las citas, se puede encontrar un tutorial en: `http://ci2.ual.es/comunicar-la-informacion/citas-y-referencias-bibliograficas/`. No obstante en las siguientes secciones se indicarán las cuestiones básicas para iniciarse en el tema pero restringuiendose a la utilización de la presente plantilla. La gran ventaja de latex es que cada elemento en las referencias `\bibitem` es considerado como un elemento de una base de datos, de forma que es el compilador el que reordena o coloca la cita de acuerdo con el estilo definido.

4.1 Como citar

A lo largo del texto de la memoria se incluirán citas a trabajos de otros autores. La inclusión de la cita tiene que ser justo después del texto donde se propone algo relativo al trabajo citado. Para citar, como para todo el LaTeX existen numerosos paquetes, los dos mas conocidos son biblatex y natbib .

Además dada forma de citar y de incorporar items en el apartado referencias lleva asociado un estilo de bibliografía `\bibliographystyle{XXX}`, que aun abre mucho mas el abanico de opciones.

Es recomendable consultar las ayudas de los paquetes `https://www.overleaf.com/learn/latex/Biblatex_bibliography_styles` y `https://www.overleaf.com/learn/latex/Bibliography_management_with_natbib`.

Lo habitual es utilizar la orden `\cite{xxx}`, o `\citep(xxx)` en la ubicación en el texto donde se quiere colocar la cita, el compilador se encargá de procesarla según el estilo.

4.2 Apartado referencias

Existen dos formas de incorporar las referencias, directamente en el entorno `\begin{bibliography}` y utilizando las ordenes del lenguaje o bien utilizando un archivo .bib para guardar esas referencias. Se generará un auxiliar .bbl que automáticamente se incorpora en el documento.

Este archivo auxiliar es el que tiene directamente las ordenes del lenguaje.

```
\begin{thebibliography}{3}

\bibitem{SWEBOK2014}
P.~Bourque and R.~E. Fairley, Eds., \emph{{SWEBOK}: Guide to the Software
  Engineering Body of Knowledge}, version 3.0~ed.\hskip 1em plus 0.5em minus
  0.4em\relax Los Alamitos, CA: IEEE Computer Society, 2014.

\bibitem{kitchenham_2013}
B.~Kitchenham and P.~Brereton, ``A systematic review of systematic review
  process research in software engineering,'' \emph{Information and Software
  Technology}, vol.~55, no.~12, pp. 2049 -- 2075, 2013.

\bibitem{basili1992}
V.~R. Basili, ``Software modeling and measurement: the goal/question/metric
  paradigm. cs-tr-2956, umiacs-tr-92-96,'' University of Maryland, Tech. Rep.,
  1992.
\end{thebibliography}
```

4.3 ARCHIVOS .BIB

BibTeX, y natbib por extensión, usan un archivo externo en texto plano como base de datos de referencias bibliográficas para generar las bibliografías y sus referencias en documentos con distintos formatos de artículos, libros, tesis, presentaciones, etc. Los nombres de archivos de referencias bibliográficas de BibTeX usualmente terminan o usan la extensión .bib. Los ítems bibliográficos incluidos en un .bib están separados por tipos.

```
@inbook{Berander2005,
        address = {Berlin, Heidelberg},
        author = {Berander, Patrik and Andrews, Anneliese},
        booktitle = {Engineering and Managing Software Requirements},
        pages = {69--94},
        publisher = {Springer Berlin Heidelberg},
        title = {{Requirements Prioritization}},
        doi= {https://doi.org/10.1007/3-540-28244-0{\_}4},
        year = {2005}
}
```

Los tipos que son reconocidos por virtualmente todos los estilos de BibTeX se muestran en el anexo B. Pero igual que antes es recomendable recurrir a tutoriales o recursos donde se describan con detalle. Además se recomienda la utilización de algún gestor de referencias bibliográficas tipo Mendeley. `https://www.mendeley.com/`

BIBLIOGRAFÍA

[1] Idealista Libertad y Control S.L., “Idealista: Portal inmobiliario en españa,” Recuperado 23 Octubre 2026 https://www.idealista.com, 2026.

[2] Airbnb Inc., “Airbnb: Alojamientos vacacionales y alquileres,” Recuperado 23 Octubre 2026 https://www.airbnb.es, 2026.

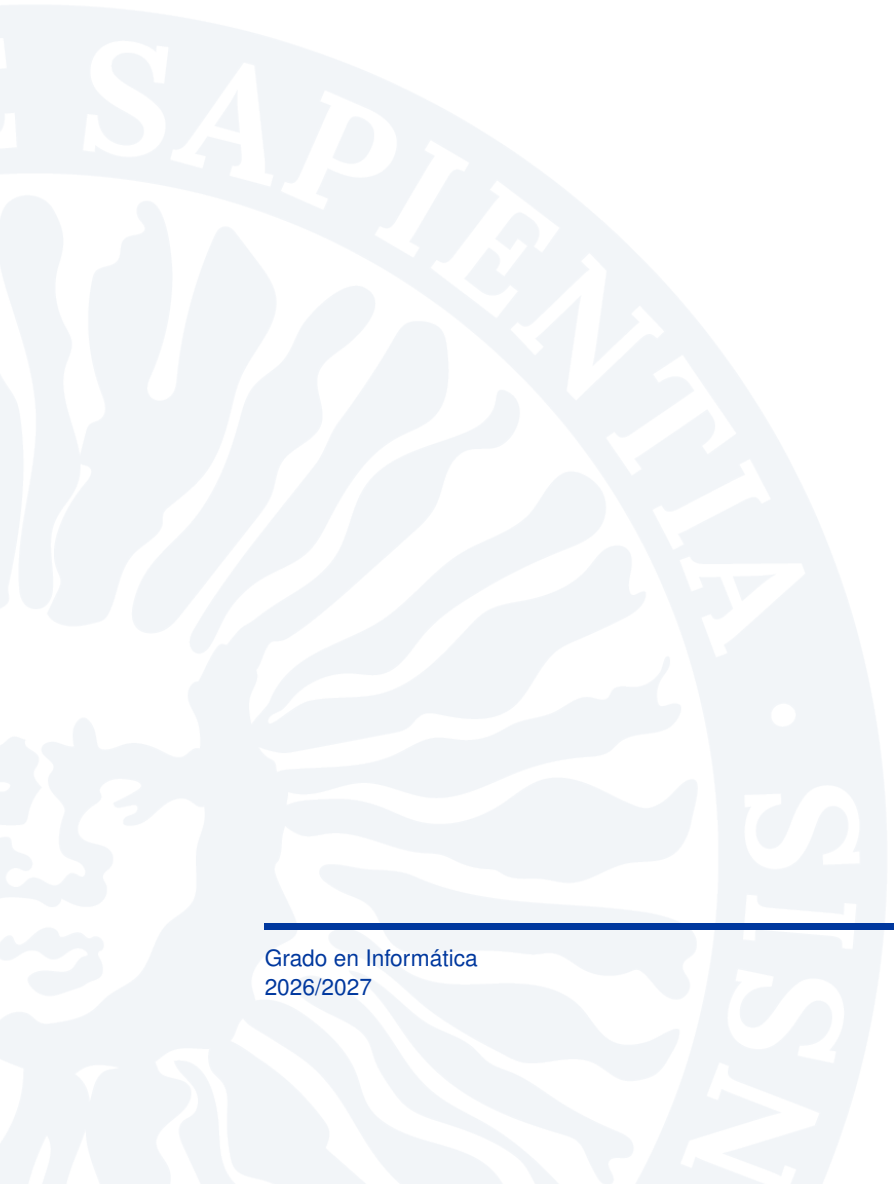
[3] Badi App S.L., “Badi: Alquiler de habitaciones y compañeros de piso,” Recuperado 23 Octubre 2026 https://badi.com, 2026.

[4] Bunq B.V., “Tricount: Gestión de gastos compartidos entre amigos,” Recuperado 23 Octubre 2026 https://www.tricount.com, 2026.

[5] Flatastic GmbH, “Flatastic: Organización doméstica y tareas de convivencia,” Recuperado 23 Octubre 2026 https://www.flatastic-app.com, 2026.

[6] P. Bourque and R. E. Fairley, Eds., *SWEBOK: Guide to the Software Engineering Body of Knowledge*. Los Alamitos, CA: IEEE Computer Society, 2014.

A Contenido de una entrada en .bib



Nombre campo	Descripción
address	Usualmente la dirección de la editorial
author	Nombre(s) del (de los) autor(es)
title	Título del libro
chapter	El número de un capítulo (o sección, etc)
edition	La edición de un libro, por ejemplo,segunda
editor	Nombre(s) del (de los) editor(es)
howpublished	Forma en que fue publicada la obra
institution	Institución responsable de un informe técnico
journal	Nombre del periódico o revista
key	Empleado para la alfabetización, referencias cruzadas y para crear una clave cuando la información del autor no está disponible. No debe confundirse con la etiqueta usada en el cite y que debe colocarse al inicio de la entrada
month	El mes de publicación o, para un trabajo inédito, en el que fue escrito
note	Cualquier información adicional que pueda ayudar al lector
number	El número del periódico, la revista, el informe técnico o del trabajo en una serie
organization	La organización responsable de una conferencia o que publica un manual
pages	Números de páginas
publisher	El nombre de la editorial. No debe confundirse con el editor
school	Nombre de la escuela donde fue escrita una tesis
series	El nombre de una serie o conjunto de libros
title	El título del trabajo
type	El tipo de un informe técnico
volume	El volumen de un periódico o una revista, o de algún libro que conste de volúmenes
year	El año de publicación. Para un trabajo inédito, el año en que fue escrito. Generalmente debe consistir de cuatro dígitos, por ejemplo 200

Tabla A.1: Campos de una entrada en archivo de bibliografía .bib

B TIPOS DE REFERENCIAS

article	Un artículo de un periódico o revista
book	Un libro con una editorial que se indica en forma explícita. Los campos requeridos en este caso son author (autor), editor, title (título), publisher (editorial) y year (año)
booklet	Una obra que está impresa y encuadernada, pero sin una editorial o institución patrocinadora
conference	Lo mismo que inproceedings, incluido para compatibilidad con el lenguaje de marcación Scribe
inbook	Una parte de un libro, que puede ser un capítulo (o sección) o un rango de páginas
incollection	Una parte de un libro que tiene su propio título
inproceedings	Un artículo en las actas de sesiones (proceedings) de una conferencia
manual	Documentación técnica
mastersthesis	Una tesis de maestría (Master thesis) o proyecto fin de carrera
misc	Para uso cuando los demás tipos no corresponden
phdthesis	Una tesis de doctorado (Ph D thesis)
proceedings	Las actas de sesiones de una conferencia
techreport	Un reporte publicado por una escuela u otra institución, usualmente numerado dentro de una serie
unpublished	Un documento que tiene un autor y título, pero que no fue formalmente publicado

Tabla B.1: Tipos posibles de elementos en bibligrafía con natbib

ARTEFACTOS SOFTWARE

UNIVERSIDAD DE ALMERÍA

se ha integrado un microservicio en Node.js que implementa el protocolo MCP (Model Context Protocol) para incorporar un asistente conversacional capaz de consultar el estado de la vivienda, recomendar anuncios o resumir la actividad mediante modelos de lenguaje.

El objetivo era consolidar buenas prácticas de ingeniería de software mediante principios SOLID y diseño guiado por el dominio (DDD), aprendiendo a resolver problemas reales de concurrencia, sincronización de datos y gamificación de tareas dando como resultado un sistema integral que cubre desde el primer contacto entre inquilino y propietario hasta el día a día dentro del hogar.

This project presents the design and implementation of Colivi, a web platform created to address common challenges in finding shared accommodations and managing household life in student flats. The main motivation behind the project stems from my personal experience during an Erasmus mobility program, where I faced significant hurdles searching for rooms from abroad, dealing with uncoordinated landlords, and managing daily domestic chores and living friction with flatmates.

To solve these issues, the system relies on a backend built with Spring Boot (Java 21) and PostgreSQL, alongside a responsive frontend developed with React. Additionally, an external Node.js microservice implementing the Model Context Protocol (MCP) was integrated to power an intelligent assistant capable of querying household chores, recommending accommodation options, and summarizing inbox activity using language models.

The primary goal was to strengthen software engineering best practices through SOLID principles and domain-driven design, learning to tackle real-world concurrency challenges, data synchronization, and gamification mechanisms. The final outcome is a comprehensive platform that supports users throughout the entire flat-sharing lifecycle, from initial room discovery to day-to-day household organization.